Multivariate Time Series Anomaly Detection with Transformer-Based Forecasting

Julian Pielmaier[1] and Lars-Alexander Böttger[1]

[1]Department of Computer Science, Leipzig University. E-mail: {jp14cola, uj74reda}@studserv.uni-leipzig.de.

Abstract
This report presents a modular framework for fully unsupervised multivariate time series anomaly detection, developed and evaluated on the Server Machine Dataset (SMD). The framework compares three detection strategies (point-wise, reconstruction-based, and forecast-based) through four models: multivariate Isolation Forest, LSTM Autoencoder, TranAD, and a custom Transformer with learnable Time2Vec embeddings built from scratch. All models share a unified detector interface and are evaluated through a multi-protocol strategy comprising point-level, point-adjust, range-level, and oracle-threshold metrics. Across 28 server machines, TranAD achieves the highest AUC-ROC of 0.92, while the custom Transformer attains 0.91 using a forecasting-based detection mechanism. The framework includes a deployment layer with a REST and WebSocket inference API, an interactive monitoring dashboard, and containerized orchestration.

1. Introduction

Detecting anomalies in time series data matters in domains from server monitoring to financial fraud detection. While univariate methods are well studied, real-world telemetry systems typically produce high-dimensional, multivariate streams where anomalies manifest as subtle inter-feature correlations rather than isolated spikes in individual channels.

This project develops a complete framework for fully unsupervised multivariate time series anomaly detection. Anomaly labels never enter the training or inference process and are used exclusively for evaluation. Detection thresholds are derived from quantiles of training-set scores, so the pipeline works without any labeled anomalies.

We compare three detection strategies. *Point-wise* methods evaluate each time step independently. *Reconstruction-based* models learn to reproduce normal patterns and flag deviations as anomalies. *Forecast-based* models predict the next time step and use the prediction error as the anomaly signal. These strategies are instantiated through four models: Isolation Forest (point-wise), LSTM Autoencoder and TranAD [1] (reconstruction-based), and a custom Transformer with learnable Time2Vec embeddings and a cross-attention decoder (forecast-based). The latter was adapted from a standalone Transformer originally developed for univariate forecasting on the Electricity Load Diagrams dataset [2], where it was also used to probe the scalability of the architecture under distributed training on GPU clusters. To ensure a fair comparison, all models share a unified pipeline with identical preprocessing, scoring, and evaluation, assessed through point-level, point-adjust, range-level, and oracle-threshold metrics alongside detection latency [3, 4].

A core part of this project was the development of a custom Transformer built from scratch, evolving from an NLP-style next-token predictor through three embedding strategies into a multivariate forecasting architecture. That this model achieves results on par with TranAD, an established benchmark, validates both the architectural choices and the embedding experiments that informed them. The multi-protocol evaluation framework proved just as important: while model rankings remained consistent

across metrics, the gap between point-level and point-adjust F1 is large enough that relying on a single protocol would give a misleading picture of detection quality.

2. Data Pipeline and Dataset Selection

2.1. From NAB to SMD

The Numenta Anomaly Benchmark (NAB) [5] is a collection of real-world and synthetic time series covering server metrics, traffic, and IoT sensor readings. Its relatively small and well-structured datasets made it a natural starting point, serving both as the testbed for the embedding experiments (Section 3.3.2) and as the testing ground for building the initial detection pipeline, where we implemented classical baselines (Rolling Z-Score, ARIMA residuals, Isolation Forest) and a univariate LSTM forecaster. The NYC Taxi dataset, with its strong and consistent periodicities (daily commuter patterns, weekend effects, rush hours, holidays), was the primary benchmark for comparing embedding strategies and verifying whether models had learned meaningful patterns. The AWS CloudWatch datasets, by contrast, record irregular server metrics such as CPU utilization. Across all model variants, training on these datasets resulted in convergence to the dataset mean, most likely because our architectures, optimized around periodic timestamp embeddings, require learnable periodic structure to be effective. We treat this as an open question and excluded these datasets from further evaluation.

As the pipeline matured, several problems with NAB became apparent. Many series contain very few annotated anomalies, and the ground-truth labels are known to be incomplete [6]. On the NYC Taxi data, small changes in thresholds led to large swings in precision and recall, making it difficult to distinguish genuine model improvements from threshold artifacts. This motivated our early adoption of range-based evaluation. More broadly, NAB's univariate structure left inter-feature dependencies unexplored.

This led us to the Server Machine Dataset (SMD) [3], which records 38 server monitoring features (CPU utilization, memory, disk I/O, network traffic, TCP statistics) across 28 machines from a large Internet company. SMD provides pre-split training and test sets with dense point-level ground-truth annotations, yielding 327 anomaly segments across the test partition. The high dimensionality, the variety of anomaly patterns, and the availability of per-feature interpretation labels made SMD better suited for controlled, multi-model evaluation. A third dataset, the Electricity Load Diagrams 2011–2014 [2], was used for the Transformer embedding and scaling experiments described in Section 3.3.

2.2. Preprocessing

SMD data is distributed as headerless text files with comma-separated values. The pipeline assigns semantic column names to all 38 features and converts the raw files into indexed DataFrames. Data is split temporally into training data and test data containing anomalies. The pipeline is designed so that any normalization is fitted exclusively on the training split to prevent information leakage. For SMD, no additional normalization is needed, since the dataset authors already pre-normalized all values to $[0, 1]$. For models requiring sequential input, sliding windows of consecutive time steps are extracted. All windowed models (LSTM Autoencoder, TranAD, and custom Transformer) use a standardized lookback of 30 time steps to ensure that differences in results reflect model architecture rather than input length, even though TranAD was originally evaluated with a window of 10 by Tuli et al. [1].

Figure 1 shows the complete pipeline from raw data through deployment. All four models share the same preprocessing and evaluation stages.

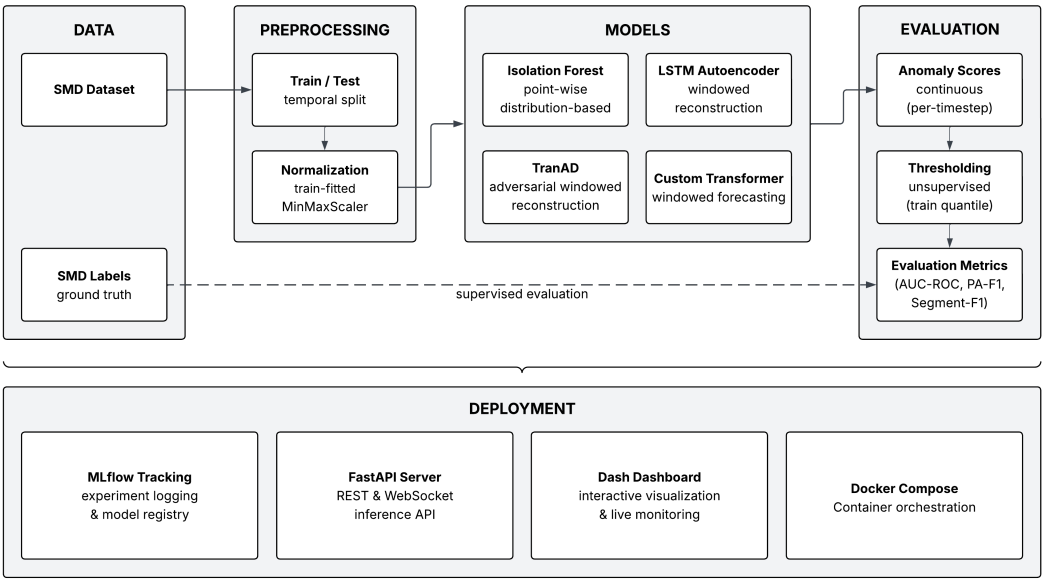

Figure 1. *End-to-end pipeline architecture. Data flows from left to right: raw SMD data is preprocessed and split temporally, then passed to four models covering three detection strategies. All models produce continuous anomaly scores that are thresholded and evaluated against ground-truth labels. Trained models and artifacts are served through a FastAPI inference server, an interactive Dash dashboard, and MLflow experiment tracking, orchestrated via Docker Compose.*

3. Model Framework

The four models represent the three detection strategies identified above. Isolation Forest serves as a point-wise baseline without temporal modeling. LSTM Autoencoder and TranAD operate on sliding windows and detect anomalies through reconstruction error, with TranAD adding adversarial self-conditioning for sharper anomaly signals. The custom Transformer takes a fundamentally different approach: it forecasts the next time step and uses the prediction error as the anomaly score. This coverage of all three strategies enables a controlled comparison of whether temporal context, and which form of it, matters for multivariate server telemetry. All models implement a common abstract interface inspired by scikit-learn, exposing `fit`, `decision_function`, and `predict` methods, which permits interchangeable use within the training pipeline, the benchmark runner, and the inference API.

3.1. Baseline Models

Multivariate Isolation Forest. The non-temporal baseline applies sklearn's Isolation Forest directly to the raw 38-dimensional feature matrix. As a point-wise method, it evaluates each time step independently without sequential context, serving as a lower bound on what temporal models should exceed.

LSTM Autoencoder. A sequence-to-sequence architecture with an encoder LSTM ($38 \rightarrow 64$), a linear bottleneck ($64 \rightarrow 32$), and a decoder LSTM ($32 \rightarrow 64 \rightarrow 38$) that reconstructs the input window. The reconstruction error (MSE per time step) serves as the anomaly score, under the assumption that anomalous patterns are poorly reconstructed by a model trained exclusively on normal data.

3.2. TranAD Reimplementation

TranAD [1] employs a Transformer encoder with two competing decoders in an adversarial self-conditioning scheme. In the first phase, the encoder processes the input window W concatenated with a zero vector, and both decoders independently reconstruct the original signal, producing outputs O_1 and O_2. In the second phase, the reconstruction error from phase one is fed back as a conditioning signal, and the second decoder produces a refined output $\hat{O}_2$. The training uses separate losses for each decoder, combining reconstruction and adversarial objectives with epoch-dependent weighting:

$$\mathcal{L}_1 = \epsilon^{-n}\|O_1 - W\|_2 + (1 - \epsilon^{-n})\|\hat{O}_2 - W\|_2 \tag{3.1}$$

$$\mathcal{L}_2 = \epsilon^{-n}\|O_2 - W\|_2 - (1 - \epsilon^{-n})\|\hat{O}_2 - W\|_2 \tag{3.2}$$

where n is the training epoch and ϵ is a parameter close to one. The first decoder minimizes the phase-two reconstruction error while the second decoder maximizes it, creating an adversarial dynamic that amplifies anomaly signals. Early in training, the reconstruction term dominates ($\epsilon^{-n} \approx 1$); as training progresses, the adversarial term takes over. Our implementation preserves the original architecture's use of custom Transformer layers without LayerNorm and with LeakyReLU activations, ensuring comparability with published results.

3.3. Custom Transformer with Time2Vec

The custom Transformer was developed as a standalone forecasting architecture before being adapted to the multivariate anomaly detection setting. This section describes the original model, the embedding experiments that shaped its design, and its adaptation to SMD.

3.3.1. Origins and Architecture Overview

The architecture originated as a next-token-prediction transformer, which was then adapted toward time series forecasting with a particular focus on how timestamps are embedded and fed into the model. While the core architecture remained relatively conventional throughout, the embedding strategy proved to be an interesting area for exploration, one where surprisingly little related work exists.

All three model variants share a common backbone. The encoder is a standard transformer encoder stack with a causal self-attention mask, ensuring that each position attends only to previous timesteps. Each input token is formed by concatenating the target value with a timestamp embedding, projected into the model dimension d_{model} via a learned linear layer. Rather than decoding from the final encoder hidden state directly, all variants use a *cross-attention* mechanism: the embedding of the target timestamp $\mathbf{t}_{fy}$ is used as the query, while the full encoder output sequence serves as keys and values. This allows the model to attend selectively to the most relevant context given *when* it is trying to predict, rather than simply reading off the last hidden state. The attended representation is then passed through a two-layer MLP decoder to produce the scalar forecast. Input dimensionality varies across datasets, since not all time series share the same timestamp resolution or feature set. To handle this, the number of timestamp features fed into the embedding (d_{tfx}, d_{tfy}) is configured per dataset. All input features, both values and timestamps, are normalized to $[0, 1]$ prior to training, as unnormalized inputs cause convergence issues.

3.3.2. Timestamp Embedding Strategies

The core question explored across three model variants was: *how can time be represented as input to a transformer?* Each variant kept the encoder-decoder structure fixed and varied the embedding approach.

Sinusoidal Embeddings. The first variant used fixed sinusoidal positional embeddings. Performance on training data was reasonable, but generalization was inconsistent. On the NYC Taxi dataset results were acceptable, but on the AWS CPU usage datasets the model consistently converged to the dataset

mean regardless of size, a clear sign that it failed to extract any temporal patterns. These datasets were set aside, with the NYC Taxi dataset becoming the primary benchmark for the embedding trials.

Time2Vec. The second variant replaced sinusoidal positional embeddings with Time2Vec [7], which encodes a scalar time value τ as a combination of a linear trend and a set of learned periodic components:

$$\mathrm{T2V}(\tau)_i = \begin{cases} \omega_i \tau + \varphi_i & i = 0 \\ \sin(\omega_i \tau + \varphi_i) & i > 0 \end{cases} \tag{3.3}$$

The formulation is similar to sinusoidal embeddings, as both use sine waves, but the key difference is that Time2Vec operates on a scalar notion of time rather than a multidimensional timestamp vector. Performance improved over the sinusoidal baseline. However, the scalar representation discards explicit calendar structure such as hour of day or day of week. Some periodicity survives implicitly, since Time2Vec is invariant to time scaling, but information that is available and likely useful is lost. This motivated a third approach.

Learnable Time2Vec with Sine and Cosine. The final variant addresses the information loss of scalar time directly. Rather than reducing timestamps to a single scalar value, the embedding takes the full normalized multidimensional timestamp vector $\mathbf{t} \in \mathbb{R}^d$ as input. The embedding is defined as:

$$v_0 = \mathbf{t}^\top \mathbf{w}_0 + b_0 \tag{3.4}$$

$$\mathbf{z} = \mathbf{W}\mathbf{t} + \mathbf{b} \tag{3.5}$$

$$\mathrm{T2V}^*(\mathbf{t}) = \left[v_0,\ \sin(\mathbf{z}),\ \cos(\mathbf{z})\right] \tag{3.6}$$

where $\mathbf{w}_0 \in \mathbb{R}^d$, $\mathbf{W} \in \mathbb{R}^{(k-1)\times d}$, and all parameters are fully learnable. The output dimension is $1+2(k-1)$, combining one linear trend term with $k-1$ sine and cosine pairs, inspired by the original Time2Vec formulation. The motivation for this design came from exposure to Discrete Fourier Transforms: the sine and cosine components form a learnable Fourier basis, allowing the model to discover data-optimized frequency representations rather than relying on fixed ones. This is particularly relevant for one of the main challenges in timestamp embeddings: unknown and varying periodicities. Our naive normalization approach feeds timestamps directly into sinusoidal embeddings, which preserves a limited notion of periodicity but introduces a structural issue at temporal boundaries. For example, 23:59 maps to a value close to 1, while 00:00 maps to 0, yielding a large distance between two timestamps that are in reality adjacent. The idea behind the periodic components $\sin(\mathbf{z})$ and $\cos(\mathbf{z})$ is to resolve this by encoding time on the complex unit circle: an angle θ is represented as $(\cos\theta, \sin\theta)$, which is continuous and wraps around by construction, with no discontinuities. The linear term v_0, by contrast, retains a non-periodic trend component for capturing long-range monotonic structure. Overall, including both sine and cosine means the embedding can represent arbitrary periodicities and the multidimensional input meanwhile preserves explicit calendar features, which a scalar encoding discards. For the NYC Taxi dataset, the encoder input $\mathbf{t}_{fx}$ carries four timestamp features (hour, quarter, weekday, day of year), while the decoder query $\mathbf{t}_{fy}$ additionally encodes a series identifier to support multi-series settings. This was more important for the LD 2011-2014 dataset, which includes multiple time series. On the NYC Taxi dataset it was just a constant.

3.3.3. Large-Scale Evaluation and Distributed Training

The ElectricityLoadDiagrams 2011–2014 dataset [2] contains electricity consumption measurements for 370 Portuguese clients, recorded at 15-minute intervals over four years. Values are given in kW, yielding approximately 140,000 time steps per client and a total dataset size of around 678 MB. The high temporal resolution, long time horizon, and large number of clients make this significantly more complex than the NAB datasets, and it served as the primary stress test for our largest transformer variant. Electricity consumption is characterized by strong and regular periodicities, daily usage cycles,

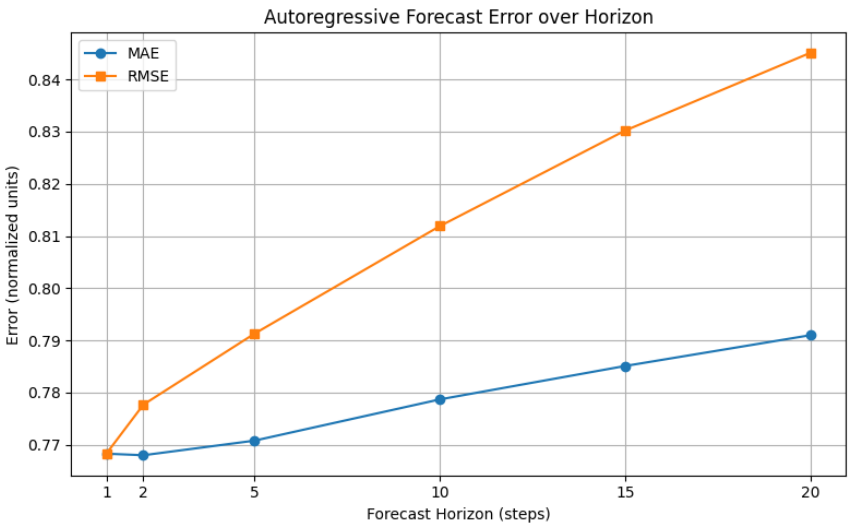

Figure 2. *Autoregressive forecast of the LD 2011-2014 dataset using a 10 million parameter transformer with Learnable Time2Vec Sine and Cosine embeddings, trained for 4 epochs with a context window of 90 timesteps (1 day).*

weekday versus weekend patterns, and seasonal trends, making it a meaningful benchmark for periodic embedding strategies. This is also the dataset where distributed training became necessary, as the window splits are not suited for anything but GPU-accelerated processing.

Training transformer models of any meaningful size requires GPU acceleration. This led us to the SC Cluster of Leipzig University, which provides multi-node GPU infrastructure. The majority of training was conducted on the Paula nodes, each equipped with 8 NVIDIA Tesla A30 GPUs. For smaller experiments this setup worked well: single-GPU training on one A30 was fast and straightforward. As we scaled to larger datasets, however, multi-GPU training became necessary, and this proved to be a time-consuming challenge. The PyTorch implementation was manageable, distributed data loaders and gradient synchronization via `torch.distributed` are well-documented. Getting this to run correctly under SLURM on the SC Cluster's specific infrastructure proved more difficult. The cluster documentation offered only guidance for basic tasks, but not specifically for distributed PyTorch jobs, and PyTorch's internal initialization timeout of 10 minutes meant that each failed configuration attempt took a significant amount of time before failing. After extended iteration, distributed training was eventually running. Even then, hardware constraints set a ceiling on what was achievable. Our largest model, at approximately 10 million parameters, could not complete 5 training epochs across 6 A30 GPUs within SLURM's 48-hour job time limit. As a result, the largest model variant was trained for 4 epochs only. It remains unclear whether this is purely a hardware constraint or the model could have been further optimized, as we only performed best-effort optimizations to iterate and test.

Figure 2 shows training results from a 10 million parameter transformer using this embedding variant. Hardware constraints limited this model to 4 training epochs. The context window was set to 90 steps, corresponding to one day of electricity usage, and the forecast horizon was predicted autoregressively. Despite the short training period, predictions remained within one standard deviation of the target, indicating that the model had extracted meaningful structure from the data.

3.3.4. Adaptation to Multivariate Anomaly Detection

In adapting this architecture to the multivariate anomaly detection setting on SMD, several modifications were necessary. The input projection was widened to accept 38 features concatenated with 16-dimensional Time2Vec embeddings, projected to $d_{\text{model}} = 64$. The Time2Vec component was reconfigured to derive time features (minute-of-hour, hour-of-day) from absolute positional indices rather than explicit calendar timestamps, since SMD lacks datetime information. The learnable multidimensional variant (Equations 3.4–3.6) was chosen based on the embedding experiments described above, with $\mathbf{t} \in \mathbb{R}^2$ mapped to a 16-dimensional embedding. The cross-attention decoder produces a 38-dimensional forecast, and the anomaly score is the MSE between the forecast and the observed next step.

Unlike TranAD's reconstruction-based scoring, this model detects anomalies via *prediction error*: if the next observed state deviates substantially from what the model expects given the recent history, the time step is flagged as anomalous. The two approaches are complementary: reconstruction measures how well a pattern fits the learned normal manifold, while forecasting measures how predictable the next state is given recent context.

4. Anomaly Scoring

4.1. Score Computation

Each model family produces anomaly scores through a different mechanism. The forecast-based custom Transformer computes the MSE between predicted and observed values. Reconstruction-based models (LSTM Autoencoder, TranAD) compute the MSE between input and reconstruction. Isolation Forest uses the negative of sklearn's decision score, derived from the average path length in the ensemble [8].

Since window-based models produce one score vector per sliding window, scores must be aggregated to the individual time step level. We use *overlap averaging*: for each time step t, the final score is the mean over all windows containing t. This approach yields smooth and stable scores.

4.2. Threshold Determination

All deep learning models determine their anomaly threshold as the q-th quantile of scores computed on training data, operating under the assumption that training data is predominantly normal. The default quantile is $q = 0.99$ for TranAD, the custom Transformer, and the LSTM Autoencoder, meaning that approximately 1% of training windows are expected to exceed the threshold. This unsupervised approach requires no labeled anomalies at training time.

5. Evaluation

5.1. Multi-Protocol Evaluation

There is no universally accepted evaluation protocol for time series anomaly detection. Point-level metrics penalize temporal misalignment harshly, while the widely used point-adjust protocol has been criticized for inflating recall [4]. We address this by computing five complementary evaluation protocols in parallel:

Point-level metrics (Precision, Recall, F1, AUC-ROC, AUC-PR) evaluate each time step independently, providing an unbiased but strict assessment.

Point-adjust (PA) metrics follow the protocol from OmniAnomaly [3]: if any point within a ground-truth anomaly segment is correctly detected, all points in that segment are credited as true positives. This reflects the operational intuition that detecting an anomaly anywhere within a segment is sufficient to trigger an alert.

Range-level metrics operate on contiguous segments rather than individual points. A ground-truth segment counts as detected if it overlaps with at least one predicted segment, yielding range precision, range recall, and range F1.

Best-F1 (oracle threshold) sweeps 100 thresholds between the 90th percentile and the maximum of test scores, selecting the threshold that maximizes point-level F1. This provides an upper bound on achievable point-level performance and quantifies how sensitive each model is to threshold selection. It is not an upper bound for PA-F1 or range-F1, which operate under different evaluation protocols.

Detection latency measures, for each ground-truth segment, the number of time steps between the segment onset and the first detection. Undetected segments are penalized with their full length.

5.2. Results

Table 1 summarizes the aggregated results across all 28 SMD machines.

Table 1. *Aggregated evaluation results across 28 SMD machines (mean ± std).*

Model	AUC-ROC	$F1_{point}$	$F1_{PA}$	$F1^{\dagger}_{point}$	Det. Rate	Latency	Inf. Time
TranAD	**.920**±.06	**.385**±.20	**.868**±.11	**.652**±.18	**86.9%**	**8.5**	3.1 ms
Custom TF (T2V)	.909±.07	.363±.18	.839±.13	.628±.19	85.0%	10.2	2.4 ms
LSTM-AE	.891±.09	.342±.20	.799±.15	.585±.20	81.9%	8.9	1.2 ms
Iso. Forest	.790±.10	.272±.15	.672±.20	.386±.18	67.9%	17.8	0.3 ms*

†Oracle-threshold not comparable to $F1_{PA}$. Latency in time steps. Inference time per window on GPU; *CPU-based.

Figure 3 complements the table by showing the per-machine AUC-ROC distribution behind these averages. The three deep learning models cluster tightly above 0.88 on most machines, with occasional outliers on particularly difficult machines. Isolation Forest, by contrast, shows a wider spread with a lower median, reflecting inconsistent performance across the dataset rather than isolated failures.

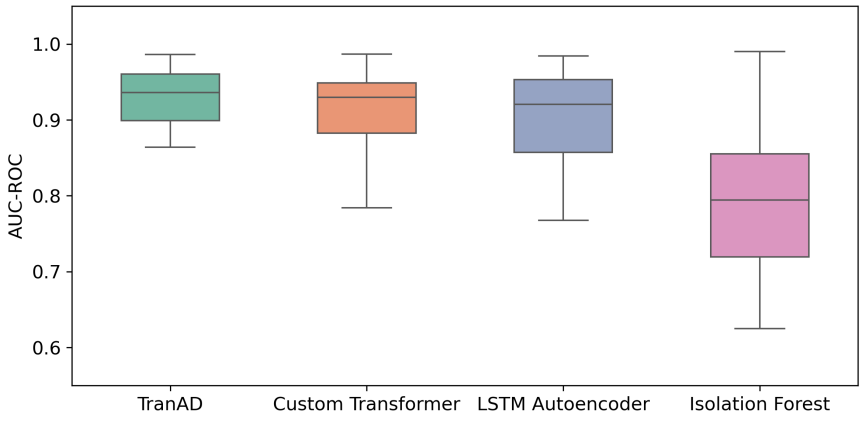

Figure 3. *AUC-ROC distribution across 28 SMD machines per model. The deep learning models achieve tight interquartile ranges above 0.88 with few low outliers, while Isolation Forest exhibits a broader spread centered around 0.79.*

5.3. Discussion

The clearest result is the gap between temporal and non-temporal methods. Isolation Forest, which evaluates each time step in isolation, consistently falls behind on every metric. Its detection rate of 67.9% and average latency of 17.8 time steps show that point-wise methods struggle to separate transient noise from sustained anomalous behavior in high-dimensional telemetry. We observed this qualitatively as well: on individual machines, Isolation Forest scores appeared noticeably noisier than those of the sequential models, with less separation between normal and anomalous regions. Figure 4 illustrates this on machine-1-1, where TranAD and the custom Transformer produce sharp, localized anomaly peaks aligned with ground-truth segments, while Isolation Forest scores are spread more diffusely.

Figure 4. *Detection results for machine-1-1, feature* `cpu_r`*. Top: time series with ground-truth anomaly regions (shaded) and per-model predictions overlaid. Bottom: corresponding anomaly scores over time. TranAD and the custom Transformer produce concentrated peaks during anomalous intervals, while Isolation Forest scores fluctuate more broadly.*

TranAD achieved the best numbers across the board, with the lowest detection latency (8.5 time steps) and the highest segment detection rate (86.9%, or 284 of 327 ground-truth segments). Tuli et al. [1] report a point-adjust F1 of 0.96 on SMD, but their evaluation uses only 4 selected machines (machine-1-1, 2-1, 3-2, and 3-7), whereas we average over all 28. They also use a window size of 10 and Peaks-Over-Threshold (POT) for threshold selection, while we use a uniform window of 30 and a fixed 99th-percentile threshold. Given these differences, particularly the subset selection, the gap between their 0.96 and our 0.87 is expected rather than indicative of an implementation issue. The self-conditioning mechanism, where the reconstruction error from a first decoder is fed back into a second decoder, appears to sharpen the anomaly signal in a way that single-pass reconstruction does not.

The custom Transformer reached results close to TranAD (AUC-ROC 0.91, PA-F1 0.84) despite using a different detection mechanism entirely. Where TranAD uses reconstruction error, the custom Transformer uses prediction error. That both approaches perform comparably suggests they capture different aspects of anomalous behavior. Looking at the score distributions, TranAD and the custom Transformer concentrate most scores near zero with a long tail into anomalous territory, while Isolation Forest produces a wider, less discriminative distribution.

The gap between point-level F1 (0.27–0.39) and point-adjust F1 (0.67–0.87) was consistent across all models. This confirms an observation we already made during our early NAB experiments: point-level metrics are strict about temporal alignment, while point-adjust inflates recall as discussed above.

Neither alone tells the full story, which is why we report both alongside range-level and oracle-threshold results. A note on reading the table: $F1^{\dagger}_{point}$ is the oracle-threshold upper bound for *point-level* F1 only, which is why PA-F1 can exceed it.

The oracle-threshold analysis shows that best-F1 scores (0.39–0.65) are considerably higher than what the default 99th-percentile threshold achieves. All models have room for improvement through better threshold calibration, and the fixed-quantile approach, while practical and fully unsupervised, leaves performance on the table.

5.4. Limitations

We evaluated on a single dataset (SMD). While SMD provides 28 machines with diverse anomaly patterns, the results may not generalize to other domains such as IoT sensor networks or financial time series.

All hyperparameters (lookback windows, learning rates, model dimensions) were set to reasonable defaults and kept constant across machines. We did not perform per-machine hyperparameter tuning, which means the reported results reflect a one-size-fits-all configuration rather than the best achievable performance for each machine. The MLflow tracking infrastructure described in Section 6 would support systematic tuning in future work.

The fixed 99th-percentile threshold is a deliberate design choice for unsupervised operation, but as the oracle-threshold analysis shows, it is far from optimal. In a production setting, a small amount of labeled data per machine could be used to calibrate thresholds, likely improving F1 scores considerably.

Finally, our custom Transformer derives time features (minute-of-hour, hour-of-day) from positional indices rather than real timestamps, since SMD does not include datetime information. On datasets with explicit timestamps, richer temporal features could be extracted, and the Time2Vec component might benefit accordingly.

6. Serving and Deployment

Beyond offline evaluation, we built a deployment layer to demonstrate that the pipeline can operate as a monitoring system, not just as an evaluation tool. A FastAPI server exposes trained models through REST endpoints for batch prediction and WebSocket endpoints for real-time streaming with pause, resume, and speed controls. An interactive dashboard built in Plotly Dash provides a system overview, per-feature data exploration with autocorrelation plots, side-by-side model comparison across machines, and a live monitoring page that streams anomaly scores in real time. MLflow tracks experiment runs and model artifacts. All three services (API, dashboard, MLflow) are containerized via a multi-stage Docker build and orchestrated through Docker Compose.

7. Conclusion

This project started with univariate anomaly detection on NAB and evolved into a modular framework for multivariate detection on SMD. The transition was driven by practical experience: working with NAB taught us that sparse, incomplete labels make fair model comparison difficult, and that threshold sensitivity is a first-order problem that no single evaluation metric can resolve. These lessons shaped the design of the final pipeline, in particular the multi-protocol evaluation and the unsupervised quantile-based thresholding.

The evaluation across 28 SMD machines shows that Transformer-based architectures, whether operating through reconstruction or forecasting, consistently outperform non-temporal baselines, detecting anomalies within 8–10 time steps of their start and completing inference in under 4 ms per window.

The fact that the forecast-based custom Transformer performs comparably to TranAD, an established benchmark model, is encouraging. A hybrid of both scoring mechanisms could be a useful next step.

Looking back, the multi-protocol evaluation was one of the more useful contributions of this work. The divergence between point-level and segment-level performance is large enough that reporting only one would give a misleading picture. The oracle-threshold analysis further showed that all models have considerable room for improvement, making adaptive thresholding a clear next step.

The custom Transformer's architecture was shaped by complementary experiments on the large-scale Electricity Load Diagrams dataset [2], which probed the scalability of the Transformer under distributed multi-GPU training and systematically compared timestamp embedding strategies, from fixed sinusoidal encodings through scalar Time2Vec to the learnable multidimensional variant. These findings directly informed the temporal encoding adopted in the multivariate pipeline presented here.

Data Availability Statement. The SMD dataset is publicly available via Kaggle (https://kaggle.com/datasets/mgusat/smd-onmiad). All code, configurations, and trained checkpoints are available in the project repository (https://github.com/Jaypee2109/multivariate-anomaly-detection). A video demonstration of the interactive dashboard is available at https://youtu.be/EwR2NSDgKig.

References

[1] Shreshth Tuli, Giuliano Casale, and Nicholas R. Jennings. TranAD: Deep Transformer Networks for Anomaly Detection in Multivariate Time Series Data. *Proceedings of the VLDB Endowment*, 15(6):1201–1214, 2022.

[2] Artur Trindade. ElectricityLoadDiagrams20112014, 2015. UCI Machine Learning Repository.

[3] Ya Su, Youjian Zhao, Chenhao Niu, Rong Liu, Wei Sun, and Dan Pei. Robust Anomaly Detection for Multivariate Time Series through Stochastic Recurrent Neural Network. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 2828–2837. ACM, 2019.

[4] Siwon Kim, Kukjin Choi, Hyun-Soo Choi, Byunghan Lee, and Sungroh Yoon. Towards a Rigorous Evaluation of Time-series Anomaly Detection. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, pages 7194–7201, 2022.

[5] Alexander Lavin and Subutai Ahmad. Evaluating Real-time Anomaly Detection Algorithms – the Numenta Anomaly Benchmark. In *Proceedings of the 14th International Conference on Machine Learning and Applications (ICMLA)*, pages 38–44. IEEE, 2015.

[6] Renjie Wu and Eamonn J. Keogh. Current Time Series Anomaly Detection Benchmarks are Flawed and are Creating the Illusion of Progress, 2020. arXiv preprint arXiv:2009.13807.

[7] Seyed Mehran Kazemi, Rishab Goel, Sepehr Eghbali, Janahan Ramanan, Jaspreet Sahota, Sanjay Thakur, Stella Wu, Cathal Smyth, Pascal Poupart, and Marcus Brubaker. Time2Vec: Learning a Vector Representation of Time, 2019. arXiv preprint arXiv:1907.05321.

[8] Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. Isolation Forest. In *2008 Eighth IEEE International Conference on Data Mining*, pages 413–422. IEEE, 2008.